

```

import queue as Q
import math

dict_hn = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 2,
    'E': 1,
    'G': 0
}

dict_gn = {
    'A': {'B': 2, 'C': 3},
    'B': {'A': 2, 'D': 5, 'E': 4},
    'C': {'A': 3, 'E': 4, 'G': 6},
    'D': {'B': 5, 'G': 3},
    'E': {'B': 4, 'C': 4, 'G': 2},
    'G': {'C': 6, 'D': 3, 'E': 2}
}

start_node = 'A'
goal_node = 'G'

def a_star_search(start, goal):
    print("--- Starting A* Search ---")
    cityq = Q.PriorityQueue()
    cityq.put((dict_hn[start], 0, start, start))

    while not cityq.empty():
        f, g, current, path = cityq.get()
        print(f"Expanding Node: {current} | Path: {path} | f(n)={f}")

        if current == goal:
            return f"{path} :: Total Cost = {g}"

        for neighbor, cost in dict_gn[current].items():
            if neighbor not in path: # Prevent loops
                new_g = g + cost
                new_f = new_g + dict_hn[neighbor]
                new_path = path + " -> " + neighbor
                cityq.put((new_f, new_g, neighbor, new_path))

    return "No path found"

class RBFSNode:
    def __init__(self, name, g, f, path):
        self.name = name
        self.g = g
        self.f = f
        self.path = path

def rbfs_search(start, goal):
    print("\n--- Starting Recursive Best-First Search ---")

    def rbfs(node, f_limit):
        if node.name == goal:
            return node, node.f

        successors = []
        for neighbor, cost in dict_gn[node.name].items():
            if neighbor not in node.path:
                g = node.g + cost
                # f(n) = max(g(n) + h(n), node.f) -> ensures f-values never decrease
                f = max(g + dict_hn[neighbor], node.f)
                successors.append(RBFSNode(neighbor, g, f, node.path + [neighbor]))

        if not successors:
            return None, math.inf

        while True:
            successors.sort(key=lambda x: x.f)
            best = successors[0]

            if best.f > f_limit:

```

```

        print(f"Backtracking from {node.name}... f-value {best.f} > limit {f_limit}")
        return None, best.f

    if len(successors) > 1:
        alternative = successors[1].f
    else:
        alternative = math.inf

    print(f"Expanding Node: {best.name} | Path: {' -> '.join(best.path)} | f(n)={best.f}")

    result, best.f = rbfs(best, min(f_limit, alternative))

    if result is not None:
        return result, best.f

    start_node_obj = RBFSNode(start, 0, dict_hn[start], [start])
    result_node, _ = rbfs(start_node_obj, math.inf)

    if result_node:
        return f"{' -> '.join(result_node.path)} :: Total Cost = {result_node.g}"
    return "No path found"

if __name__ == '__main__':
    a_star_result = a_star_search(start_node, goal_node)
    print(f"\n[FINAL A* RESULT] Optimal Path: {a_star_result}")

    rbfs_result = rbfs_search(start_node, goal_node)
    print(f"\n[FINAL RBFS RESULT] Optimal Path: {rbfs_result}")

```

```

--- Starting A* Search ---
Expanding Node: A | Path: A | f(n)=6
Expanding Node: B | Path: A -> B | f(n)=6
Expanding Node: C | Path: A -> C | f(n)=7
Expanding Node: E | Path: A -> B -> E | f(n)=7
Expanding Node: E | Path: A -> C -> E | f(n)=8
Expanding Node: G | Path: A -> B -> E -> G | f(n)=8

[FINAL A* RESULT] Optimal Path: A -> B -> E -> G :: Total Cost = 8

--- Starting Recursive Best-First Search ---
Expanding Node: B | Path: A -> B | f(n)=6
Expanding Node: E | Path: A -> B -> E | f(n)=7
Backtracking from E... f-value 8 > limit 7
Backtracking from B... f-value 8 > limit 7
Expanding Node: C | Path: A -> C | f(n)=7
Expanding Node: E | Path: A -> C -> E | f(n)=8
Backtracking from E... f-value 9 > limit 8
Backtracking from C... f-value 9 > limit 8
Expanding Node: B | Path: A -> B | f(n)=8
Expanding Node: E | Path: A -> B -> E | f(n)=8
Expanding Node: G | Path: A -> B -> E -> G | f(n)=8

[FINAL RBFS RESULT] Optimal Path: A -> B -> E -> G :: Total Cost = 8

```

```

import matplotlib.pyplot as plt

graph = {
    'MVLU': {'MIDC': 12, 'Tolani': 14},
    'MIDC': {'SEEPZ': 6, 'Marol': 10},
    'Tolani': {'Marol': 13},
    'SEEPZ': {'Chandivali': 9},
    'Marol': {'Chandivali': 6},
    'Chandivali': {'Powai_Lake': 3},
    'Powai_Lake': {}
}

heuristics = {
    'MVLU': 22, 'MIDC': 14, 'Tolani': 13,
    'SEEPZ': 9, 'Marol': 7, 'Chandivali': 2, 'Powai_Lake': 0
}

coords = {
    'MVLU': (2, 6), 'MIDC': (1, 4.5), 'Tolani': (3, 4.5),
    'SEEPZ': (1, 3), 'Marol': (3, 3), 'Chandivali': (2, 1.5), 'Powai_Lake': (2, 0)
}

def rbfs_search(start, goal):
    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'), path=[start])

```

```

        return path, cost

def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return True, path, g, g

    neighbors = graph.get(node, {})
    if not neighbors:
        return False, [], 0, float('inf')

    successors = []
    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            next_g = g + distance
            next_f = max(next_g + heuristics[neighbor], g + heuristics[node])
            successors.append([next_f, neighbor, next_g])

    if not successors:
        return False, [], 0, float('inf')

    while True:
        successors.sort(key=lambda x: x[0])
        best = successors[0]

        if best[0] > f_limit:
            return False, [], 0, best[0]

        alternative_f = successors[1][0] if len(successors) > 1 else float('inf')

        success, result_path, total_g, returned_f = rbfs(
            best[1], goal, best[2], min(f_limit, alternative_f), path + [best[1]]
        )

        best[0] = returned_f
        if success:
            return True, result_path, total_g, returned_f

path, total_dist = rbfs_search('MVLU', 'Powai_Lake')
print(f"Optimal RBFS Path: {' -> '.join(path)} ({total_dist} km)")

plt.figure(figsize=(8, 7))
for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]
        is_path = node in path and neighbor in path and path.index(neighbor) == path.index(node) + 1
        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)
        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width, zorder=1)
        plt.text((x1+x2)/2, (y1+y2)/2, f"{dist}km", color='red', fontsize=9, ha='center')

for node, (x, y) in coords.items():
    color = '#f1c40f' if node in path else '#3498db'
    plt.scatter(x, y, color=color, s=700, zorder=2)
    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center', va='center', color='white', fontsize=8, fontweight='bold')

plt.title(f"RBFS Shortest Route Map (Total: {total_dist}km)", fontsize=12, fontweight='bold')
plt.axis('off')
plt.show()

```

Show hidden output

```

import matplotlib.pyplot as plt
import queue as Q

graph = {
    'MVLU': {'MIDC': 12, 'Tolani': 14},
    'MIDC': {'SEEPZ': 6, 'Marol': 10},
    'Tolani': {'Marol': 13},
    'SEEPZ': {'Chandivali': 9},
    'Marol': {'Chandivali': 6},
    'Chandivali': {'Powai_Lake': 3},
    'Powai_Lake': {}
}

heuristics = {
    'MVLU': 22, 'MIDC': 14, 'Tolani': 13,
    'SEEPZ': 9, 'Marol': 7, 'Chandivali': 2, 'Powai_Lake': 0
}

```

```

}

coords = {
    'MVLU': (2, 6), 'MIDC': (1, 4.5), 'Tolani': (3, 4.5),
    'SEEPZ': (1, 3), 'Marol': (3, 3), 'Chandivali': (2, 1.5), 'Powai_Lake': (2, 0)
}

def a_star_search(start, goal):
    open_list = Q.PriorityQueue()
    open_list.put((heuristics[start], 0, start, [start]))

    while not open_list.empty():
        f_score, g_score, current_node, path = open_list.get()

        if current_node == goal:
            return path, g_score

        neighbors = graph.get(current_node, {})
        for neighbor, distance in neighbors.items():
            if neighbor not in path:
                new_g = g_score + distance
                new_f = new_g + heuristics[neighbor]

                new_path = list(path)
                new_path.append(neighbor)
                open_list.put((new_f, new_g, neighbor, new_path))

    return [], 0

path, total_dist = a_star_search('MVLU', 'Powai_Lake')
print(f"Optimal A* Path: {' -> '.join(path)} ({total_dist} km)")

plt.figure(figsize=(8, 7))
for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]
        is_path = node in path and neighbor in path and path.index(neighbor) == path.index(node) + 1
        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)
        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width, zorder=1)
        plt.text((x1+x2)/2, (y1+y2)/2, f"{dist}km", color='red', fontsize=9, ha='center')

for node, (x, y) in coords.items():
    color = '#f1c40f' if node in path else '#3498db'
    plt.scatter(x, y, color=color, s=700, zorder=2)
    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center', va='center', color='white', fontsize=8, fontweight='bold')

plt.title(f"A* Shortest Route Map (Total: {total_dist}km)", fontsize=12, fontweight='bold')
plt.axis('off')
plt.show()

```

Show hidden output